Tema 6
Análisis estático de código

Verificación y validación de Software

Contenidos

1. Motivación.
2. Técnicas de análisis estático.
3. Herramientas e integración en el desarrollo.

1. Motivación

Detectando problemas aún sin ejecutar el código.

1. Motivación – Definición

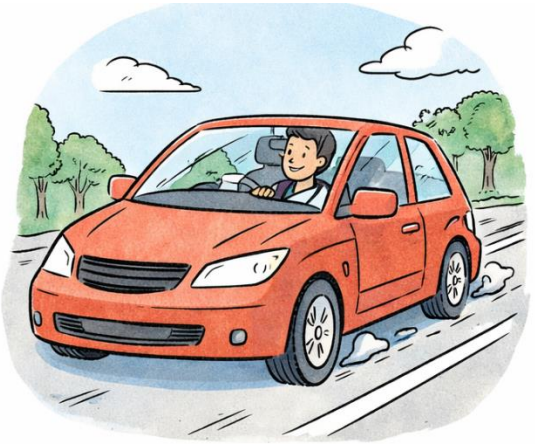
El **análisis estático** hace referencia a las técnicas para razonar sobre el comportamiento de un programa sin ejecutarlo.

Un **analizador estático de código** es un programa que analiza y razona sobre el comportamiento de otros programas.

Técnica de verificación estática

- El programa no se ejecuta.
- Es diferente al Testing.
  (que es una técnica de verificación dinámica)

Verificación estática



Verificación dinámica

1. Motivación – Definición

Con **análisis estático de código** podemos prevenir:

- **Problemas de programación** que pueden dar lugar a errores.
  Variables no inicializadas, mal manejo de nulos, …
- **Vulnerabilidades** en nuestro código
  inyección de SQL, no validar entradas, …
- **Malas prácticas** de programación.
  Code smells, …
- Integración con **métricas de testing**.
  Cobertura, complejidad, deuda técnica, …

Diferentes herramientas:

SonarQube, Gitlab SAST, Github CodeQL, …

1. Motivación – Ejemplo

¿Es correcto este código?

```
class Persona {
  private Integer edad;

  public int getEdadEnMeses() {
    return edad * 12;
  }
}
```

1. Motivación – Ejemplo

¿Es correcto este código?

- El error puede venir de los datos de entrada.
- Aunque el testing nos puede ayudar a detectar el problema, un simple vistazo al código nos habría servido.

```
class Persona {
  private Integer edad;

  public int getEdadEnMeses() {
    return edad * 12;
  }
}

Persona persona = new Persona();
int edad = persona.getEdadEnMeses();
```

La edad no se ha inicializado.

```
Exception in thread "main"
java.lang.NullPointerException:Cannot invoke
"java.lang.Integer.intValue()" because "this.edad" is null at
Persona.getEdadEnMeses(Persona.java:5)
```

1. Motivación – Anotaciones

Algunos lenguajes tienen **mecanismos de anotación** que se usan para añadir información adicional de manera cómoda.

- Las anotaciones se pueden usar para facilitar el análisis del código sin ejecutarlo.
- Indican restricciones.
  Por ejemplo, *este valor no puede ser null*.
- Mayor capacidad expresiva.
  El tipo de dato no siempre expresa todas las restricciones.
- El IDE y herramientas de análisis estático usan esta información.

```java
import javax.annotations.NonNull;

class Persona {
  private @NonNull Integer edad;

  public int getEdadEnMeses() {
    return edad * 12;
  }
}
```

1. Motivación – ¿Qué errores se detectan automáticamente?

Cuanta más información tenga el código, más errores podemos detectar.

Ejemplo:

Los tipos de datos ya hacen análisis estático.

Tipado dinámico.

- No se asumen restricciones sobre los valores.
- Los tipos solo existen en tiempo de ejecución.

Tipado estático.

- Detectan errores en tiempo de compilación.
- Pero no son infalibles…
- No siempre expresas todas las restricciones (como en la nulabilidad).
- No expresan todas las restricciones del dominio.

```
x = "hola"
x = 10
print (x + " mundo")
```

Error en tiempo de ejecución

```
int x = 10;
x = "hola";
```

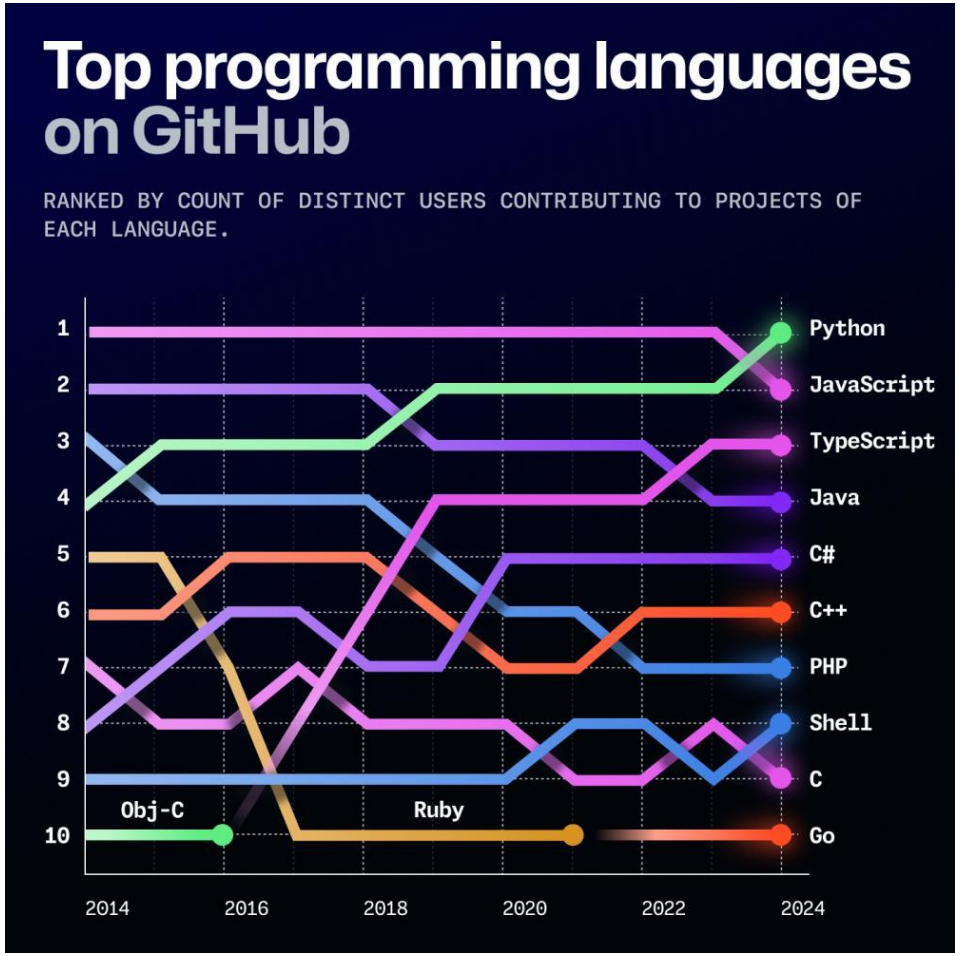
Error en tiempo de compilación

1. Motivación – Lenguajes graduales

- **Dinámicos** (Javascript, PHP, ...)
  El tipo se comprueba en ejecución.
  Más robustos, pero poco flexibles y verbosos.
- **Estáticos** (Java C#, C++, C, ...)
  El tipo se comprueba en compilación.
  Flexibles, pero propensos a errores en ejecución.

Lenguajes modernos permiten añadir tipos de forma opcional **(lenguajes graduales)**.

```
def saludar(nombre: str) -> str:
  return "Hola" + nombre;
```



1. Motivación – Limitaciones del análisis estático

	Análisis estático	Análisis dinámico
¿Qué necesita?	Solo el código fuente. (no hacen falta tener datos de entrada)	Ejecutar el programa. (con datos de prueba concretos)
¿Qué analiza?	Tiene que imaginar todos los casos posibles.	Sólo observa lo que ocurre en una ejecución concreta.
¿Qué detecta?	Puede avisar de errores que no ocurren (falsos positivos).	Los errores que detecta son reales (verdaderos positivos).
¿Qué omite?	Trata de cubrir todos los posibles problemas. (aunque no todos sean reales)	Puede no detectar errores que sí existen. (falsos negativos).

1. Motivación – Beneficios

El análisis estático no es perfecto, pero sí que es muy útil.

Detecta errores antes de ejecutar.

- Variables no inicializadas.
- `NullPointerException`
- Información confidencial expuesta.

Mejora el código

- Detección de código muerto y duplicado.
  Elimínalo y reduce el tamaño del binario.
- Complejidad innecesaria.
  Por ejemplo, sacar expresiones constantes de un bucle.

Entender el programa.

- ¿Cuáles son los sitios del programa donde se invoca a la función f?
- ¿Qué partes están conectadas?
- ¿Qué código no se usa?

1. Motivación – Usos

El análisis estático mejora la **calidad del código:**

- Encuentra fallos pequeños antes de que se conviertan en grandes.
- Sugiere refactorizaciones.
- Mantiene convenciones de código y evita discusiones en el equipo *(tabs vs spaces)*
- Hace el código menos complejo y más fácil de mantener.
- Identifica *code smells*.
- Hace la calidad visible: *métricas, cobertura, deuda técnica, …*
- Encuentra problemas de usabilidad y accesibilidad.
- Identifica cuellos de botella y mejora el rendimiento.

1. Motivación – Code smells

Código que funciona… pero huele mal:

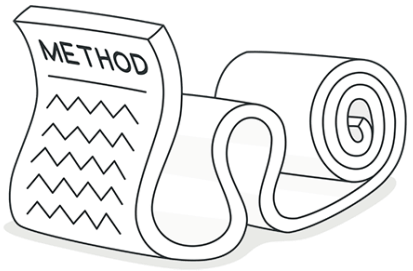
- No es un error. **¡El programa funciona!**.
- Es señal de baja calidad o mal diseño.
- Pero indica problemas futuros.

Taxonomía https://refactoring.guru/refactoring/smells

- **Bloaters:** Código demasiado grande.
- **Change Preventers:** Código difícil de cambiar.
- **Dispensables:** Código innecesario o redundante.
- **Couplers:** Código demasiado acoplado.

```
public void procesarPedido (Pedido p) {
  // validar datos
  …

  // calcular precio
  …

  // aplicar descuentos
  …

  // guardar en base de datos

  …
  // enviar email
  …
}
```

code smell
Long method



1. Motivación – Refactoring

- Modificar el código sin cambiar su comportamiento.
- Mejora la legibilidad, estructura y calidad.

Estrategias

- Los code smells indican dónde refactorizar.
- Los tests automatizados garantizan que el comportamiento se mantiene.

Técnicas

- Renombrar variables y métodos.
- Separar responsabilidades.
- Reorganizar el código.

```
public void procesarPedido (Pedido p) {
  // validar
  …

  // procesarPedido
  …

  // notificar
  …
}
```

2. Técnicas de análisis estático

Estrategias para razonar sobre el código.

2. Técnicas – Herramientas

- **Linters** *(estilo)*
  Estilo y convenciones de código.
- **Detectores de errores basados en patrones** *(bugs comunes)*
  Problemas simples basadas en reglas.
- **Procesadores de anotaciones** *(más información)*
  Extiende el sistema de tipos.
- **Análisis de flujo de datos** *(interpretación abstracta y más profunda)*
  Análisis profundo del comportamiento del programa.

2. Técnicas – Linters

- Analizan el código de forma rápida y superficial.
- Comprueban estilo y convenciones. Garantizan que el código tiene un estilo estandarizado siguiendo guías de estilo.
- Mejoran la mantenibilidad.
  - Dedicamos más tiempo a leer y comprender el código.
- No detectan errores complejos.

Ejemplos de guía de estilos

- Asegurar indentación adecuada.
- Convenciones de nombres.
- Tamaños de líneas (ej., < 80)
- Anidamiento de clases.
- Documentación en todas las funciones públicas.
- Paréntesis en las expresiones.

2. Técnicas – Patrones

- Detectan problemas comunes en el código
- Se basan en reglas conocidas (patrones)
- No analizan en profundidad, pero son muy efectivos.

Ejemplos de problemas que detectan.

- Malas prácticas
- Corrección
- Rendimiento
- Internacionalización
- Código malicioso
- Corrección Multihilo
- Seguridad
- Código sospechoso (code smells, dodgy code)

Ejemplo:
https://spotbugs.readthedocs.io/en/latest/bugDescriptions.html

SpotBugs

2. Técnicas – Patrones – Malas prácticas

```java
String x = new String("Foo");
String y = new String("Foo");

if (x == y) {
  System.out.println("¡iguales!");
} else {
  System.out.println("¡diferentes!");
}
```

```java
String x = new String("Foo");
String y = new String("Foo");

if (x.equals(y)) {
  System.out.println("¡iguales!");
} else {
  System.out.println("¡diferentes");
}
```

Bad Practice
ES_COMPARING_STRINGS_WITH_EQ Comparing strings with ==

2. Técnicas – Patrones – Rendimiento

```java
public static String repeat(
  String string,
  int times
) {
  String output = string;
  for(int i = 1; i < times; ++i) {
    output = output + string;
  }
  return output;
}
```

```java
public static String repeat(
  String string,
  int times
) {
  StringBuilder output = new StringBuilder(
    string.length() * times
  );
  for (int i = 1; i < times; ++i) {
    output.append(string);
  }
  return output.toString();
}
```

Performance

SBSC_USE_STRINGBUFFER_CONCATENATION. Method concatenates strings using + in a loop.

Un String es inmutable, por lo que cada vez que se concatena se crea una nueva cadena. Si se hace en un bucle se van a crear múltiples copias de la misma cadena.

2. Técnicas – Patrones – Corrección

```
public DbConnection getConexion() throws SQLException {
  DbConnection con = new DbConnection(host, port);

  try {
    con.open();
  } catch (Exception e) {
    new SQLException(e);
  }

  return con;
}
```

```
try {
  con.open();
} catch (Exception e) {
  throw new SQLException(e);
}
```

Missing `throw` before `new Exception`

2. Técnicas – Patrones – Ejemplo

Proyecto análisis-estático

https://github.com/cs-umu/pds-2026/tree/main/ejemplos/analisis-estatico

Analizador estático que busca el *smell* de no abrir un `closeable` en un ***`try`***

Estrategia:

- Procesar una clase Java con `Java Parser`.
- Explorar el árbol de sintaxis:

1. En cada método, buscar ocurrencias de creación de un objeto
2. Comprobar si el objeto se ha creado dentro de un try

* Generado con Claude Sonnet 3.7 – con correcciones a mano

2. Técnicas – Anotaciones

- Las anotaciones añaden información adicional al código.
- Permiten expresar restricciones que el tipo no indica.
- Cada lenguaje tiene su propio sistema de anotaciones.
  Ejemplo: Java con `@anotación`
- Existen analizadores específicos para procesar anotaciones.

2. Técnicas – Anotaciones – Nulabilidad

Anotaciones de **nulabilidad**.

`@Nullable`

- Indica que un valor puede ser `null`.
- Obliga a comprobar su valor antes de usarlo.

```
// Una persona puede no tener apodo
@Nullable
String getApodo(Persona p) {
  return persona.getApodo ();
}
```

```
// Usos
// INCORRECTO (posible NullPointerException)
String apodo = getApodo(persona);
int longitud = apodo.length();

// Correcto
String apodo = getApodo(persona);
if (apodo != null) {
  int longitud = apodo.length();
}
```

2. Técnicas – Anotaciones – Nulabilidad

Anotaciones de **nulabilidad**.

@NonNull

- Indica que un valor no debe ser `null`
- Permite a herramientas detectar usos incorrectos.

```
int calcularEdad(@NonNull String fecha) {
  …
}
```

```
// Uso
// INCORRECTO
// (pero atrapado por herramientas)
calcularEdad (null);
```

2. Técnicas – Anotaciones – Nulabilidad

```
import org.checkerframework.checker.nullness.qual.*;

public class NullnessExampleWithWarnings {
   public void example() {
      @NonNull String foo = "foo";
      String bar = null;
      foo = bar;
      println(foo.length());
   }
}
```

@Nullable se aplica por defecto

Error: [assignment.type.incompatible] incompatible types in assignment.
found : @Initialized **@Nullable** String
required: @UnknownInitialization **@NonNull** String

2. Técnicas – Anotaciones – Unidades

Existen anotaciones para identificar unidades.

```
@Time
private Long timeToLive = 5000L * UnitsTools.s;

@Time
private Long extraTime = 1500L * UnitsTools.s;

@Time
public Long extraTTL(){
  return timeToLive + extraTime;
}
```

https://dzone.com/articles/improving-code-quality-with-checker-framework-1

2. Técnicas de análisis estático – Anotaciones

- Ejemplo: unidades

```
import static org.checkerframework.checker.units.UnitsTools.m;
import static org.checkerframework.checker.units.UnitsTools.mPERs;
import static org.checkerframework.checker.units.UnitsTools.s;

void demo() {
  @m int x;
  x = 5 * m;

  @m int meters = 5 * m;
  @s int seconds = 2 * s;

  @mPERs int speed = meters / seconds;
  @m int foo = meters + seconds;
  @s int bar = seconds - meters;
}
```

`@m` indicates that `x` represents meters

To assign a unit, multiply appropriate unit constant from `UnitTools`

3. Herramientas e integración

Automatizando la calidad en el software.

3. Herramientas – Automatización

En el desarrollo tradicional, la calidad depende del desarrollador.

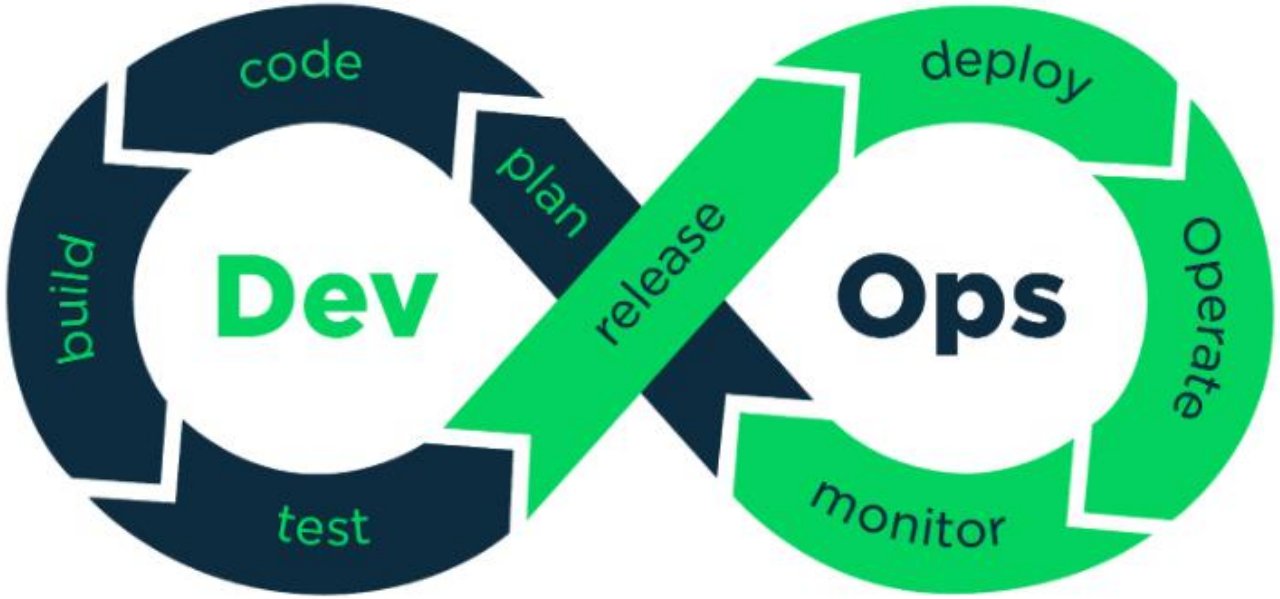
- Muchas tareas son manuales y repetitivas.
- Cada uno trabaja de forma distinta.
- Es fácil olvidarse de pasos importantes.

¿Quién garantiza que se ejecuten las pruebas, que el proceso compile y que se revise la calidad?

3. Herramientas – DevOps

DevOps es un enfoque de desarrollo moderno.

- Une desarrollo (Dev) y operación (Ops).
- Se basa en colaborar y automatizar.
- Busca entregar el software más rápido, reducir errores y mejorar la fiabilidad.



3. Herramientas – CI / CD

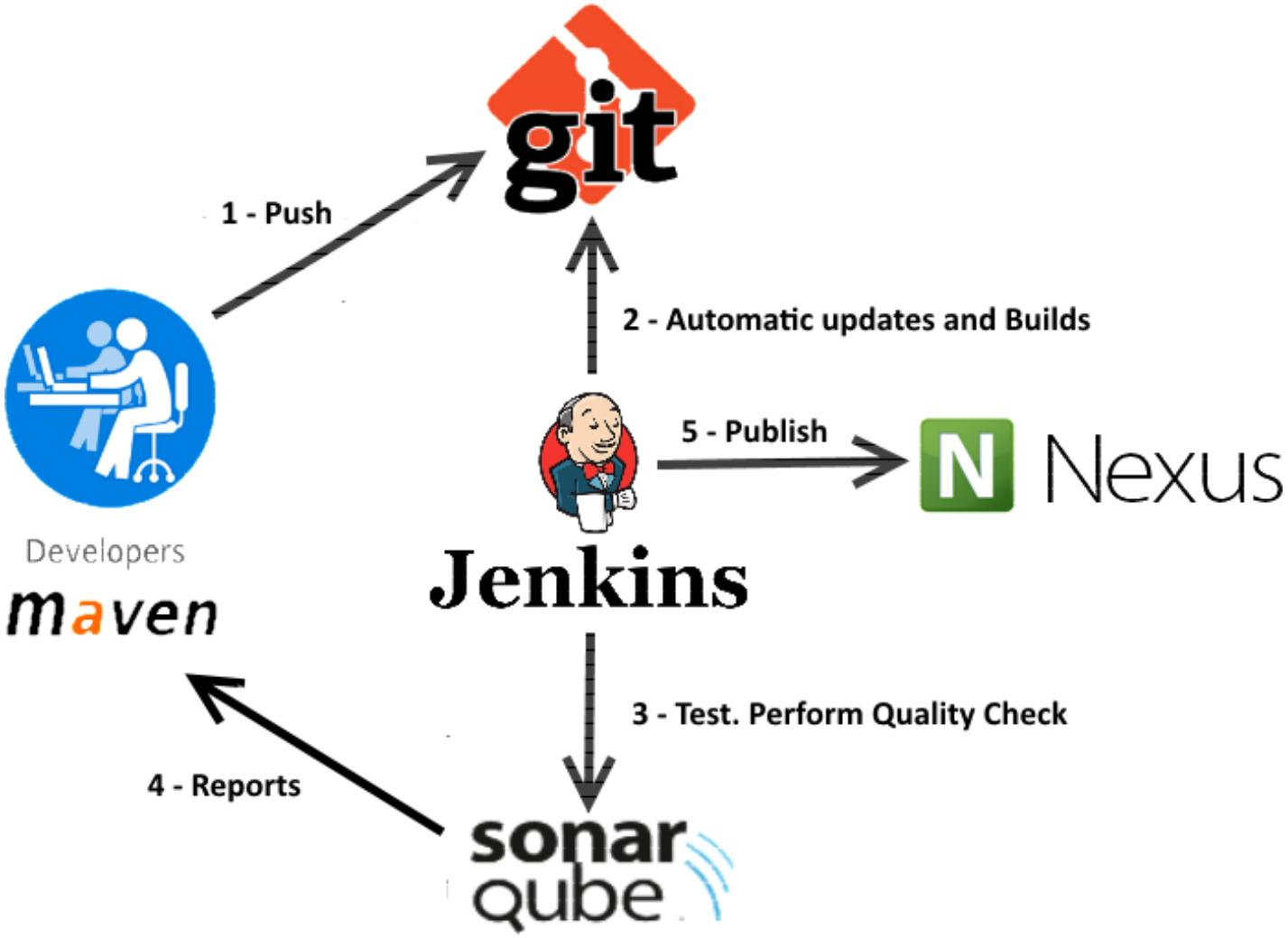
Integración continua (CI)

- Los desarrolladores suben código a un repositorio compartido.
- Cada cambio dispara:
  compilación, suite de pruebas, análisis estático, …
- Objetivo:
  detectar errores pronto.

Entrega continua (CD)

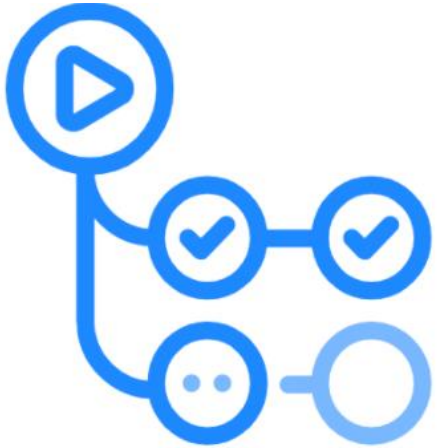
- Tras las pruebas, el software se prepara y despliega automáticamente.
- Se prepara para su despliegue (producción, pruebas, etc.)
- Objetivo:
  entregar software pronto.

3. Herramientas – Integración continua



3. Herramientas – Integración continua – Github Actions

- GitHub Actions es una plataforma de CI/CD.
- Automatiza el proceso de construcción, pruebas y despliegue.
- Crea flujos de trabajo (workflows)
  - Para construir y probar un *pull requests*.
  - Para desplegar un *pull request* que se ha integrado.
  - En general, realiza acciones como respuesta a eventos en el repositorio.



GitHub Actions

3. Herramientas – Integración continua – Github Actions

```yaml
name: tests
on: push
jobs:
  run_tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the repository
        uses: actions/checkout@v2
      - name: Set up JDK 11
        uses: actions/setup-java@v1
        with:
          java-version: 11
      - name: Run tests with Maven
        run: mvn -B test --file pom.xml
```

Nombre del trabajo.

Contenedor Docker.

Descarga el código del contenedor.

Configura la JVM del contenedor.

Compila y ejecuta los tests con Maven.

Cualquier valor de retorno distinto de 0 se considera un fallo de CI.

3. Herramientas – Integración continua – Github Actions

Despliegue.

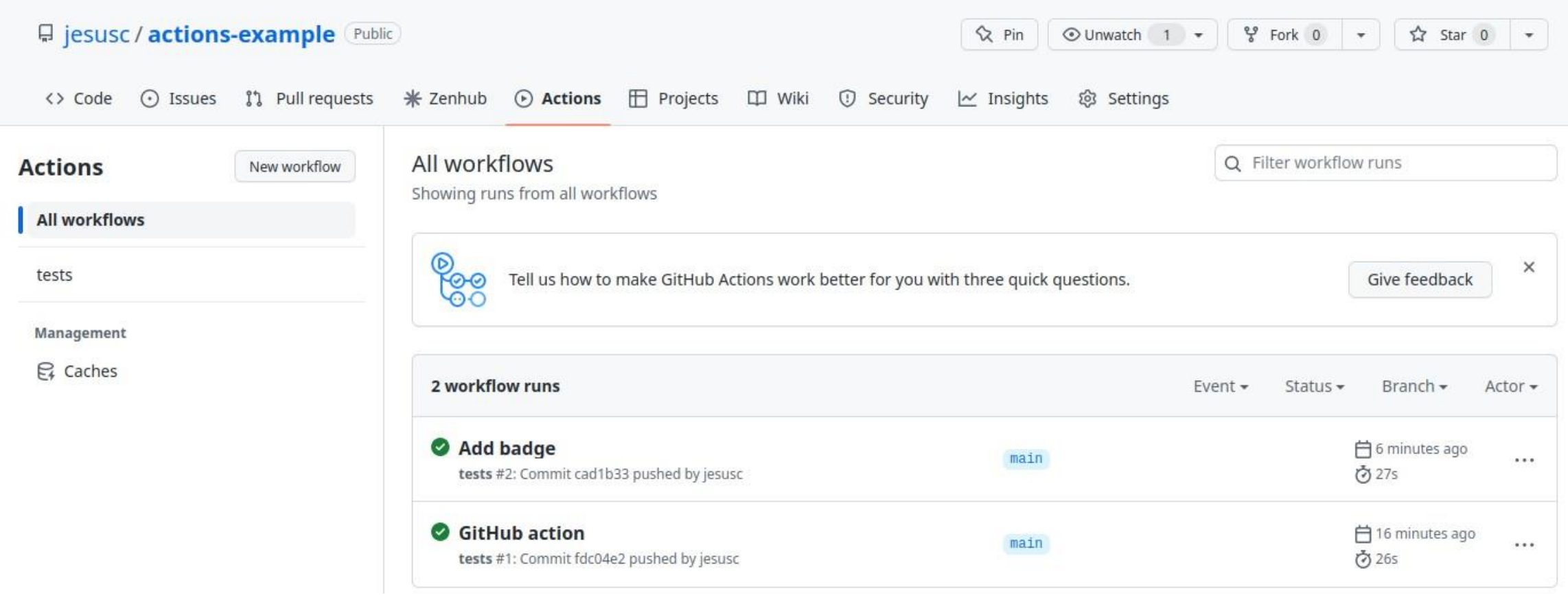
https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-java-with-maven

```
steps:
- uses: actions/checkout@v3
- uses: actions/setup-java@v3
  with:
    java-version: '11'
- run: mvn --batch-mode --update-snapshots verify
- run: mkdir staging && cp target/*.jar staging
- uses: actions/upload-artifact@v3
  with:
    name: Package
    path: staging
```

Copiar y desplegar artefactos.

3. Herramientas – Integración continua – Github Actions

Comprobar el resultado



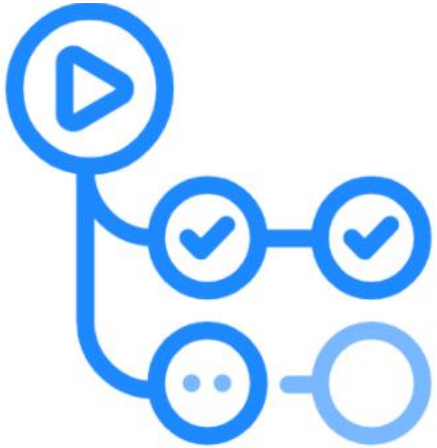
Añadir un badge

```
![example workflow](https://github.com/jesusc/actions- example/actions/workflows/check-junit.yaml/badge.svg)
```

3. Herramientas – Integración continua – Github Actions

¿De dónde obtener las acciones?

- Built-in
- Marketplace
  https://github.com/marketplace?type=actions
- **Ejemplos**
  - https://github.com/marketplace/actions/close-stale-issues
  - https://github.com/marketplace/actions/prettier-java-action
- Se pueden definir acciones propias.



GitHub Actions

3. Herramientas – SonarQube

Plataforma de análisis estático (SAST).

- Analiza código sin ejecutarlo.
- Permite detectar Bugs, Vulnerabilidades, Code smells, …
- Visión global del proyecto (no solo reglas individuales).
- Diferentes métricas de calidad.
  - Maintainability.
  - Technical Debt.
- Permite análisis en local ya que se integra con:
  - Eclipse
  - Visual Studio Code
  - IntelliJ

3. Herramientas – SonarQube

- Disponemos de un servidor SonarQube para la asignatura.
- Puede integrarse con Maven en cualquier ciclo CI/CD.

Más documentación

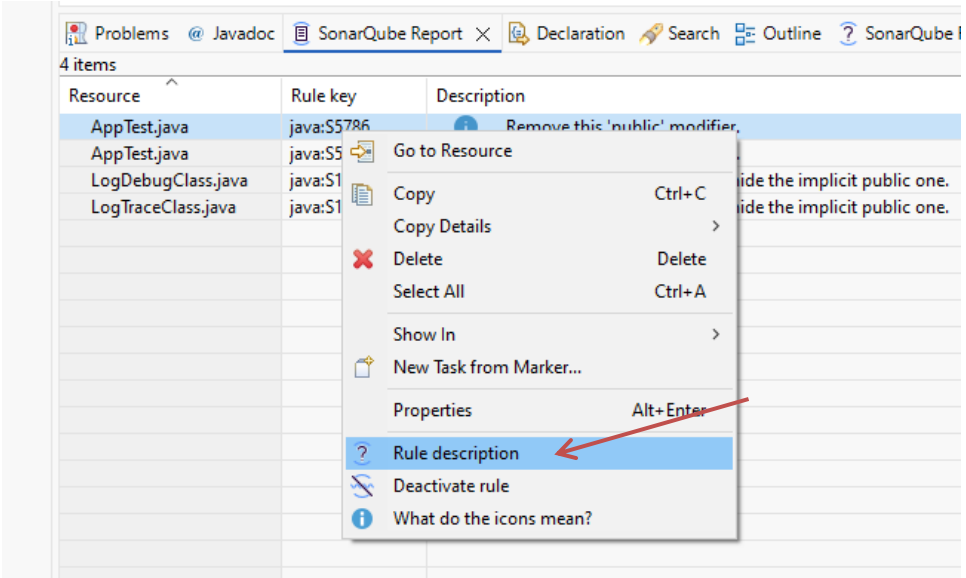
https://github.com/cs-umu/pds-2026/blob/main/practicas/transparencias/logging.pdf

3. Herramientas – SonarQube – Ejemplo

Problems | Javadoc | SonarQube Report | Declaration | Search | Outline | SonarQube Rule Description | Console | SonarQube On-The-Fly

4 items

Resource	Rule key	Description	Date
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
LogDebugClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago
LogTraceClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago



3. Herramientas – SonarQube – Ejemplo

Why is this an issue? | How can I fix it? | More Info

A reference to `null` should never be dereferenced/accessed. Doing so will cause a `NullPointerException` to be thrown. At best, such an exception will cause abrupt program termination. At worst, it could expose debugging information that would be useful to an attacker, or it could allow an attacker to bypass security measures.

Note that when they are present, this rule takes advantage of nullability annotations, like `@CheckForNull` or `@Nonnull`, defined in JSR-305 to understand which values can be null or not. `@Nonnull` will be ignored if used on the parameter of the `equals` method, which by contract should always work with null.

Why is this an issue? | How can I fix it? | More Info

Noncompliant code example

The variable `myObject` is equal to `null`, meaning it has no value:

```
public void method() {
  Object myObject = null;
  System.out.println(myObject.toString()); // Noncompliant: myObject is null
}
```

The parameter `input` might be `null` as suggested by the `if` condition:

```
public void method(Object input)
{
  if (input == null)
  {
    // ...
  }
  System.out.println(input.toString()); // Noncompliant
}
```

4. Análisis de la arquitectura

con ArchUnit

4. ArchUnit

¿Qué es ArchUnit?

- Librería open-source para verificar la arquitectura de aplicaciones Java
- Permite comprobar la arquitectura de nuestro proyecto lanzándose como un test unitario
- Características:
  - Integrado con Maven
  - Inspecciona el código binario de las clases compiladas
  - Contiene un amplio abanico de reglas predefinidas
  - Permite crear nuestras propias reglas
  - Permite generar nuestros propios mensajes de error

```
<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit5</artifactId>
  <version>1.4.1</version>
  <scope>test</scope>
</dependency>
```

4. ArchUnit

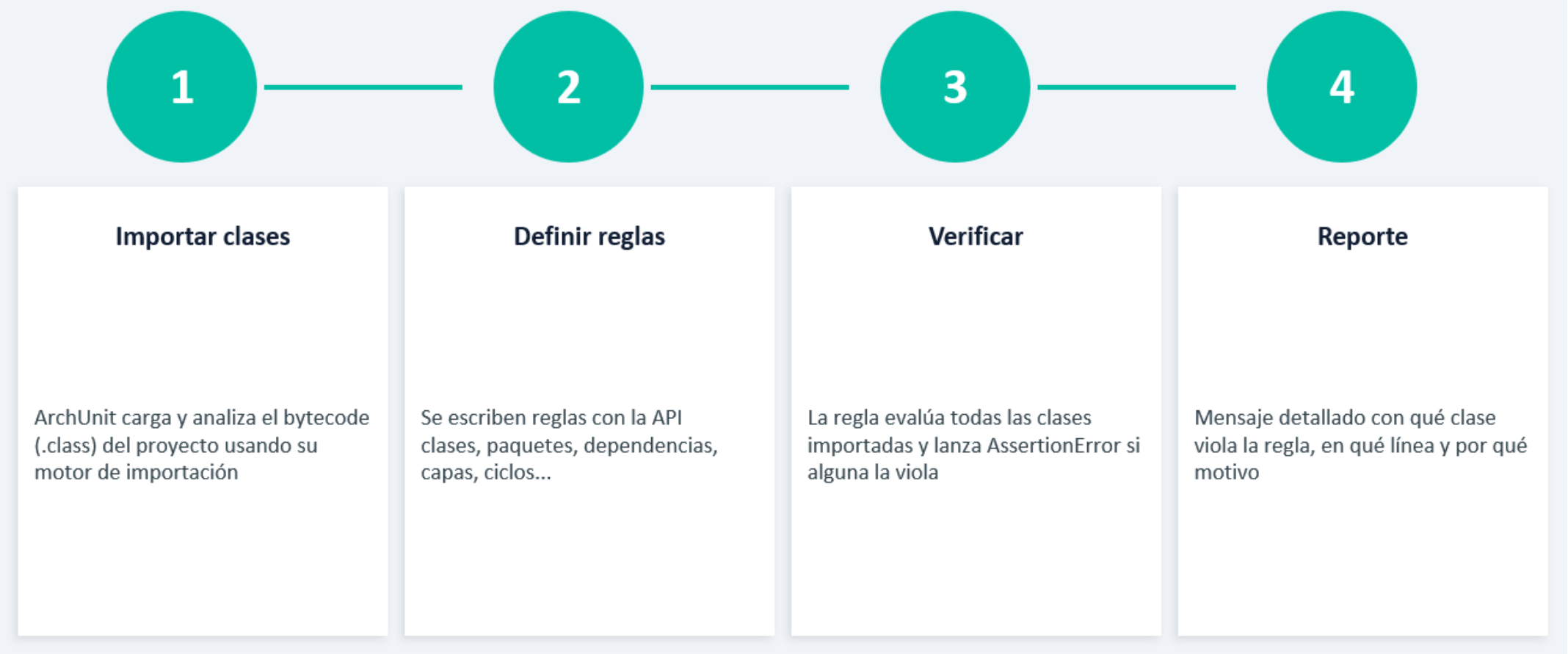
Detectar antes de tener que corregir

ArchUnit nos permite crear reglas para

- Mantener una correcta separación de capas
- Evitar un alto acoplamiento entre módulos de nuestra aplicación
- Evitar acumular código autogenerado por el IDE
- Conseguir un nombrado coherente de nuestras clases

4. ArchUnit

Proceso



4. ArchUnit

¿Cómo se compone una regla ArchUnit?

SUJETO	VERBO	RAZÓN

`classes()`
`.that()`
`.resideInAPackage`
`("..service..")`
	`.should()`
	`.beAnnotatedWith`
	`(@Service.class)`
		`.because("Los servicios`
		`deben estar anotados`
		`con @Service")`

¿Sobre qué clases aplica la regla?	¿Qué condición deben cumplir?	Mensaje en el error

4. ArchUnit

Creación de mi primer test

Cuando creo un test ArchUnit es recomendable usar la anotación @AnalyzeClasses para limitar los paquetes que quiero que analice

```
@AnalyzeClasses(packages =“pds.umulingo")
public class UmuLingoArquitecturaTests { .. }
```

Con este código se restringe el análisis a las clases que están dentro del paquete `pds.umulingo` o hijos

Cada test debe tener la anotación @ArchTest

- El estándar de nombrado es poner las palabras separándolas por guiones bajos
- Definiremos una variable estática y a la derecha de la asignación la ejecución de las reglas que dará como salida una ArchRule

```
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = …
```

4. ArchUnit

Creación de mi primer test

Una vez que defino mi primera regla tengo que seleccionar los elementos con los que quiero operar

ArchUnit por defecto provee métodos para:

- Regla que deben cumplir todas las clases classes()
- Regla que no debe cumplir ninguna clase noClasses()
- Regla que deben cumplir todos los métodos methods()
- Regla que no debe cumplir ningún método noMethods()
- …

```
@ArchTest
static final ArchRule paquete_raiz1 = allClasses().should().resideInAPackage("pds.umulingo..");

// Alternativa
@ArchTest
static final ArchRule paquete_raiz2 = noClasses().should().resideOutsideOfPackage("pds.umulingo..");
```

Todas las reglas iniciales podemos consultarla en la clase ArchRuleDefinition

Importante: En la documentación normalmente se importan esos métodos de manera estática, como otras reglas.

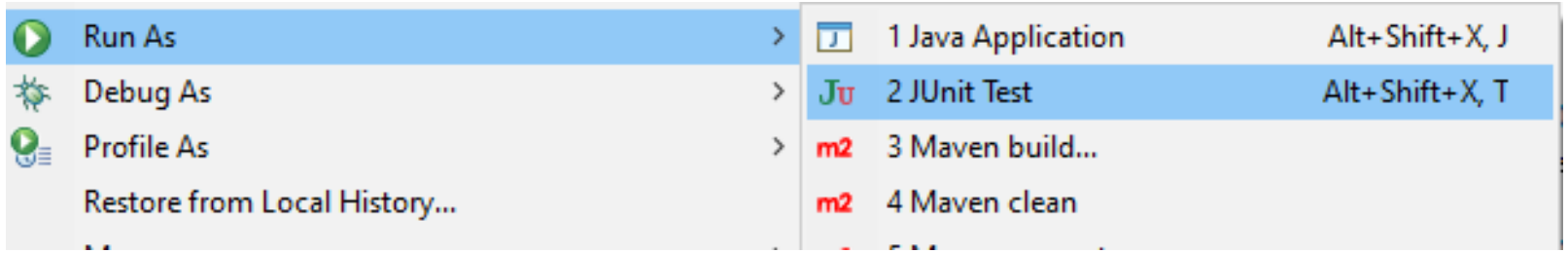
- Normalmente en los ejemplos no se muestran los imports
- Si nuestro código no compila al copiar los ejemplos debemos revisar que nuestros imports sean correctos

```
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.methods;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
```

4. ArchUnit

Creación de mi primer test

Para ejecutar los test es suficiente lanzarlos como un test unitario



4. ArchUnit – Arquitectura Hexagonal

- **Regla básica**
  - La capa de dominio no puede acceder a los elementos de la infraestructura
- **Definición de capas**
  - Dominio: Núcleo de la aplicación, incluye definición de los puertos
  - Aplicación: implementación de los servicios (puertos de entrada, *driving* ports)
  - Adaptadores: implementación de los puertos de salida o *driven* ports

4. ArchUnit – Arquitectura Hexagonal

```
@ArchTest
static final ArchRule codigo_respeta_arquitectura_hexagonal = layeredArchitecture()
                // Todas las dependencias del proyecto
                .consideringAllDependencies()
                // Defino la capa de dominio como las clases dentro del paquete domain
                .layer("Domain").definedBy("..domain..")
                // Defino la capa de aplicacion = paquete aplicacion
                .layer("Application").definedBy("..application..")
                // Adaptadores = clases dentro de adapters
                .layer("Adapters").definedBy("..adapters..")
                // Indico que "imports" de la
                // capa de dominio pueden estar en Application y Adapters
                .whereLayer("Domain").mayOnlyBeAccessedByLayers("Application", "Adapters")
                // Indico que "imports" de la capa de
                // adaptadores pueden estar en Application (pero no en dominio)
                .whereLayer("Application").mayOnlyBeAccessedByLayers("Adapters")
                // Ninguna capa debe tener import de adapters (lo gestiona Spring Boot)
                .whereLayer("Adapters").mayNotBeAccessedByAnyLayer();
```

Es necesario adaptarlo a cada proyecto concreto

Bibliografía

- Software Engineering – Basic Principles and Best Practices. Ravi Sethi. Cambridge University Press. 2023.
  - Capítulo 8